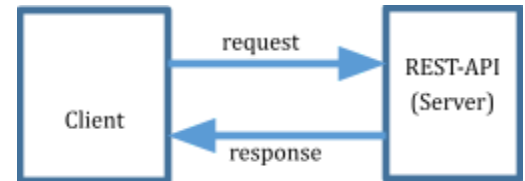


Getting Started with FastAPI

What is an API?

- **API:** Application Programming Interface (API) is a contract between an information provider and an information user that enables the communication between the two. Here information provider and user both are a kind of software and they don't have to be in the same language



- REST (Representative State Transfer) API is a type of contract. A set of rules that both parties adhere to.
REST-API has the following characteristics:

- Client-Server (like browser-web server)
- Has URI (like web server): Uniform resource identifier
- Stateless (doesn't remember your last call)

If we want our application to be available for other developers, creating an API acts as an intermediate connector. Developers will send HTTP requests to this API to consume the service. Users don't need the code or install dependencies. They only call this API through HTTP requests. It also simplifies integration, making it easier to integrate with third-party solutions. Before using an API, we must know the APIs, HTTP methods, and error codes.

HTTP Methods

- **GET**
Retrieve data from the server (read-only, no modification).
Example: Fetch user details or product list.
- **POST**
Send data to the server to create a new resource.
Example: Submit a form, create a new user.

- **PUT**
Update or replace an existing resource completely.
Example: Update full user profile.
 - **PATCH**
Partially update an existing resource.
Example: Change only email or password.
 - **DELETE**
Remove a resource from the server.
Example: Delete a record.
-

HTTP Status Code Categories

- **2xx – Success**
Request processed successfully.
Examples:
 - **200 OK** – Request succeeded
 - **201 Created** – Resource created
- **3xx – Redirection**
Client must take further action.
Examples:
 - **301 Moved Permanently**
 - **302 Found**
- **4xx – Client Error**
Request has an issue from the client side.
Examples:
 - **400 Bad Request** – Invalid input
 - **401 Unauthorized** – Authentication required
 - **404 Not Found** – Resource missing
- **5xx – Server Error**
Server failed to process a valid request.
Examples:
 - **500 Internal Server Error**



- 503 Service Unavailable
-

FastAPI

FastAPI is a modern Python framework for building APIs quickly. It's popular because it:

- Is **fast** (high performance)
- Has **automatic API docs** (Swagger UI)
- Supports **type hints** to validate inputs and outputs
- Makes it easy to build clean, production-ready APIs

Typical use cases:

- Backend for web/mobile apps
- Microservices
- AI/ML model serving APIs
- Internal tools and automation services

In the context of Generative AI, FastAPI is used to turn AI models, prompt pipelines, RAG systems, and multi-agent workflows into secure, scalable web APIs that can be accessed by applications, automation tools, and enterprise systems.

It acts as the backend layer that receives user inputs, validates requests, triggers LLMs or agents, and returns AI-generated responses in real time. By providing high performance, async support, and automatic documentation, FastAPI helps move GenAI solutions from experimental notebooks to reliable, production-ready services that can power chatbots, copilots, analytics tools, and business automation platforms.

Prerequisites

You should have:

- Python 3.10+
 - Basic Python knowledge (functions, imports)
 - Command line basics
-



IDE to Use

An IDE (*Integrated Development Environment*) is a software application that provides a complete workspace that helps developers write, test, debug, and manage code efficiently in one place.

When building a FastAPI application, you can use any Python-supported IDE such as **VS Code** (Visual Studio Code), **PyCharm**, etc.

VS Code is widely preferred because it is lightweight, free, and offers powerful extensions for Python, debugging, virtual environment management, REST client testing, and Git integration.

VS Code needs to be installed on your system before you can use it for development. Alternatively, you can use **GitHub Codespaces** as well, which offers the user interface similar to VS Code, managed by GitHub, and is accessible in the browser.

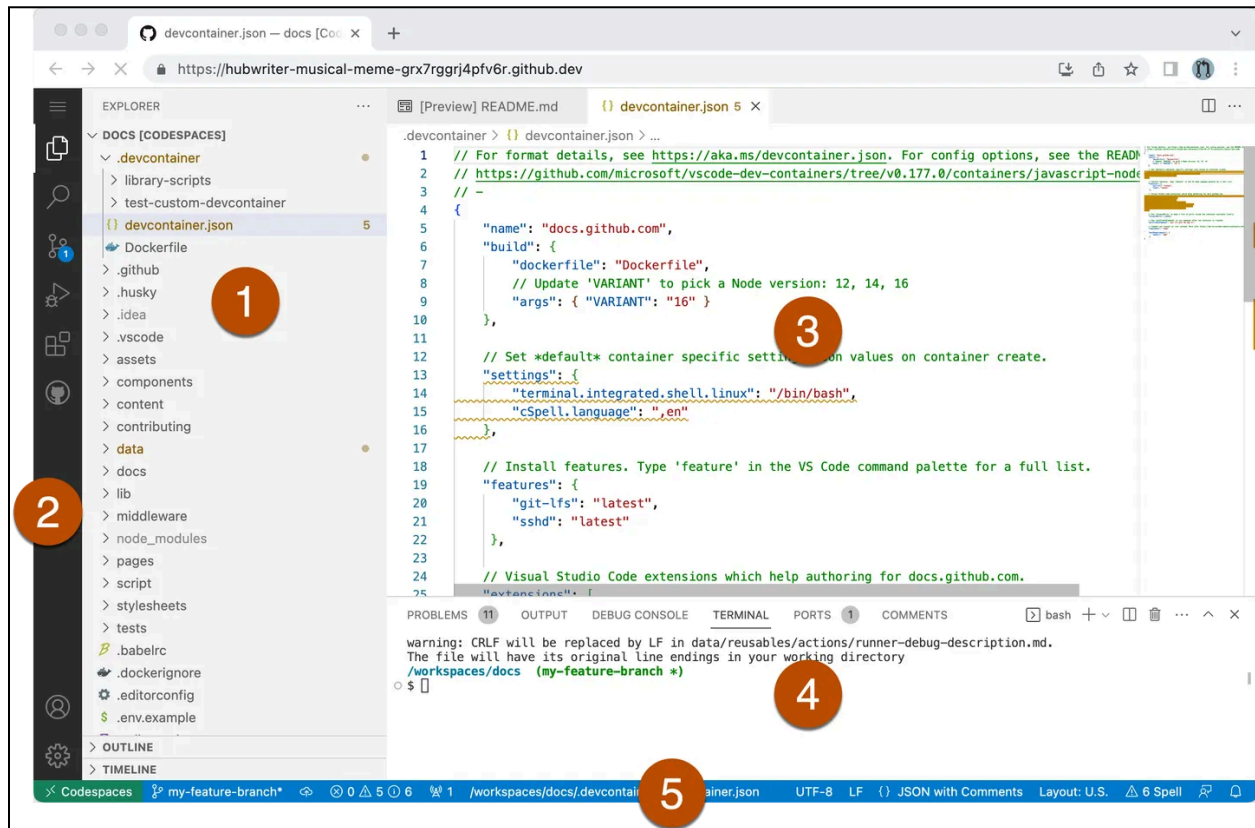
What is a GitHub Codespace?

A codespace is a development environment that's hosted in the cloud.

Features:

- Each codespace you create is hosted by GitHub in a Docker container, running on a virtual machine.
- You can choose from a selection of virtual machine types, from 2 cores, 8 GB RAM, and 32 GB storage, up to 32 cores, 64 GB RAM, and 128 GB storage.
- You can work in a codespace using your browser.

The main components of the user interface are shown below:



1. **Sidebar** - shows your project files in the Explorer
2. **Activity bar** - provides you with a way to switch between the Views
3. **Editor** - this is where you edit your files
4. **Panels** - this is where you can see output and debug information, as well as the default place for the integrated Terminal
5. **Status bar** - provides you with useful information about your codespace and project. For example, the git branch name, configured ports, and more

About billing for GitHub Codespaces

GitHub Codespaces is paid for either by an organization, an enterprise, or a personal account. The Free and Pro plans for personal accounts include **free use** of GitHub Codespaces **up to a fixed amount** of usage every month.

For personal accounts, the Codespaces are free for 120 hours per month.

Free quota

All GitHub personal accounts include a quota of free compute time and storage for GitHub Codespaces. Any usage beyond the included amounts is billed to the personal account.

Account plan	Storage per month	Compute time per month
GitHub Free for personal accounts	15 GB-month	120 hrs
GitHub Pro	20 GB-month	180 hrs

To know more about billing for GitHub codespaces, refer [here](#).

Build Your First FastAPI App

Here, we will build a FastAPI application using GitHub Codespaces (a VS Code equivalent interface provided by GitHub).

1. Go to GitHub and login: <https://github.com/>
(Create a new GitHub account if you don't have an existing one)
2. Access GitHub Codespaces by going to this page: <https://github.com/codespaces>
3. Start a Blank codespace by going to **Blank** >> **Use this template**

Your codespaces

Explore quick start templates

Blank
By github 

Start with a blank canvas or import any packages you need.

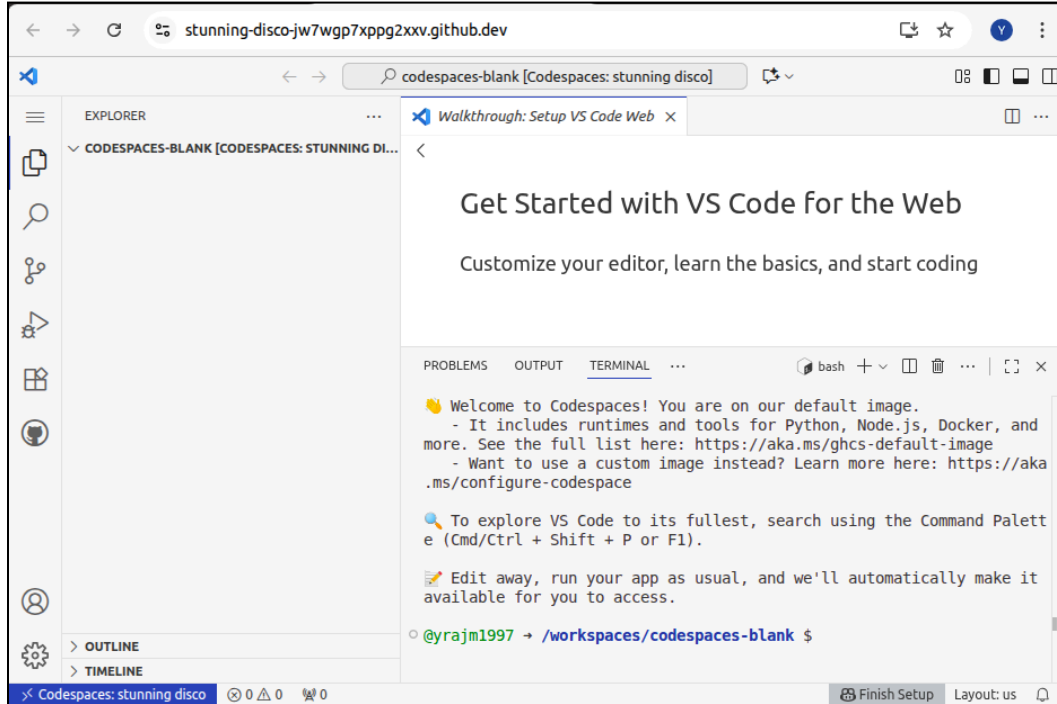
Use this template

React
By github 

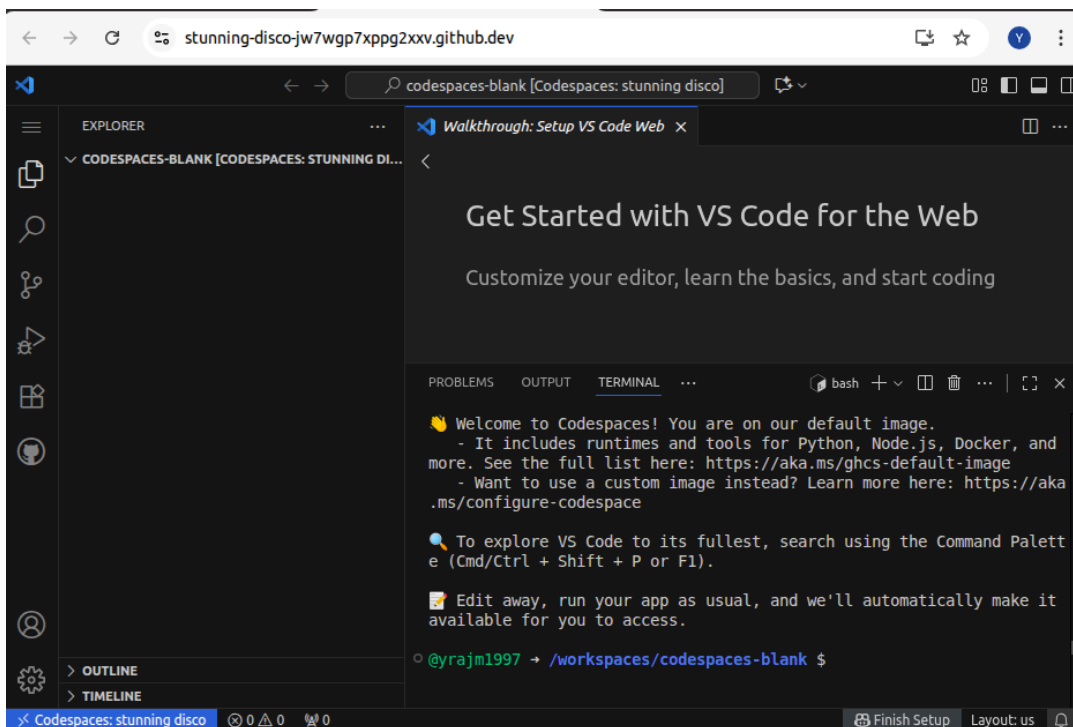
A popular JavaScript library for building user interfaces based on UI components.

Use this template

4. Your Codespace should be created.



5. Once the Codespace is created, you can change its color theme as per your choice by going to **Settings icon** on bottom-left >> **Theme** >> **Color Theme** >> **Dark Modern**





6. In Codespace, Python comes preinstalled. Check the Python version installed using the below command:

```
python --version
```

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
● @yrajm1997 → /workspaces/codespaces-blank $ python --version
Python 3.12.1
```

Virtual Environment

A *Virtual Environment* is an isolated workspace in Python that allows you to install and manage project-specific libraries without affecting other projects or the global Python installation on your system.

7. Run the below commands to create a Virtual Environment:

```
python -m venv venv
```

Here, the **first venv** is the python module name, and **second venv** is the name given to the virtual environment.

```
● @yrajm1997 → /workspaces/codespaces-blank $ python -m venv venv
○ @yrajm1997 → /workspaces/codespaces-blank $
```

8. Once the environment is created, run the below command to activate it:

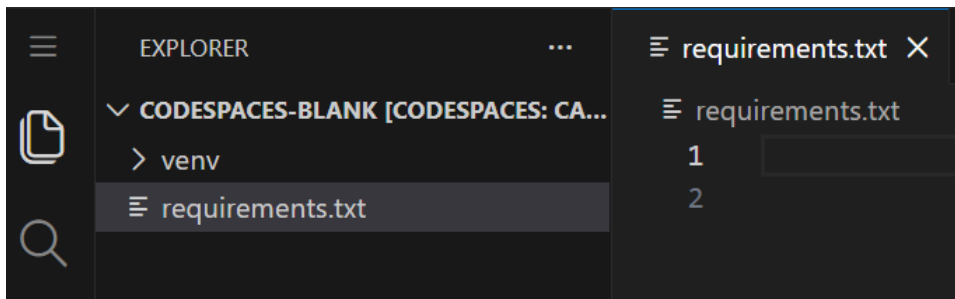
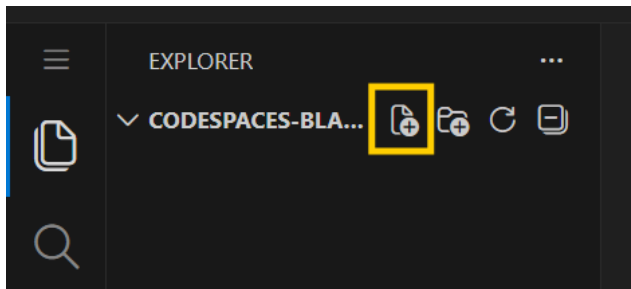
```
source venv/bin/activate
```

```
● @yrajm1997 → /workspaces/codespaces-blank $ python -m venv venv
● @yrajm1997 → /workspaces/codespaces-blank $ source venv/bin/activate
○ (venv) @yrajm1997 → /workspaces/codespaces-blank $
```

Install FastAPI + Uvicorn (server)

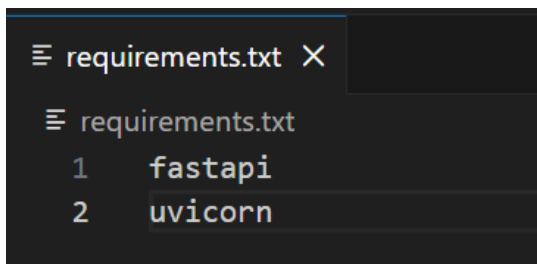
Uvicorn is a web server used to run FastAPI applications and handle incoming client requests efficiently. It acts as the bridge between users and your FastAPI app by listening for HTTP requests, managing connections, and passing requests to the application for processing.

9. Click on New file icon and add a new file with name **requirements.txt**
Make sure the file is created outside the venv folder.



10. Add below libraries to the **requirements.txt** file:

```
fastapi
uvicorn
```



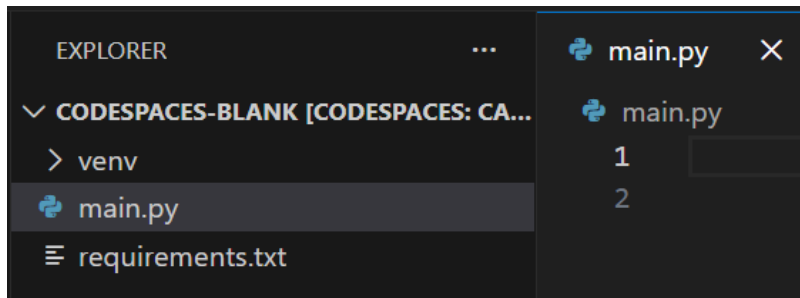
11. Run the below command to install required libraries:

```
pip install -r requirements.txt
```

```
(venv) @yrajm1997 → /workspaces/codespaces-blank $ pip install -r requirements.txt
Installing collected packages: uvicorn, fastapi
Successfully installed fastapi-0.128.7 uvicorn-0.40.0

[notice] A new release of pip is available: 23.2.1 -> 26.0.1
[notice] To update, run: pip install --upgrade pip
(venv) @yrajm1997 → /workspaces/codespaces-blank $
```

12. Click on New file icon again and add a new file with name **main.py**



13. Add the below content to the **main.py** file.

```
Python
# Import FastAPI
from fastapi import FastAPI

# Create FastAPI object
app = FastAPI()

# Add a GET method to the "/" endpoint
@app.get("/")
def root():
    return {"message": "Hello from FastAPI!"}

# Import Uvicorn (Web server)
import uvicorn

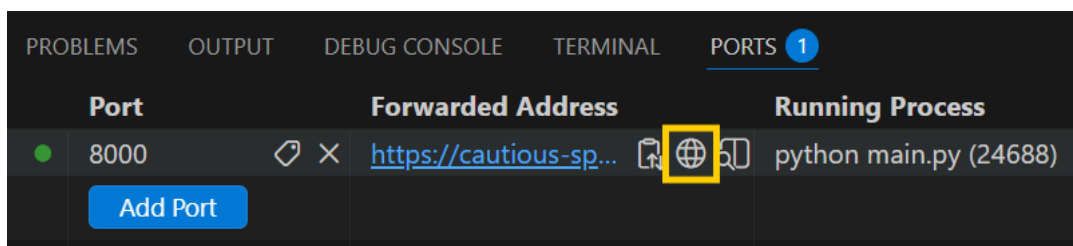
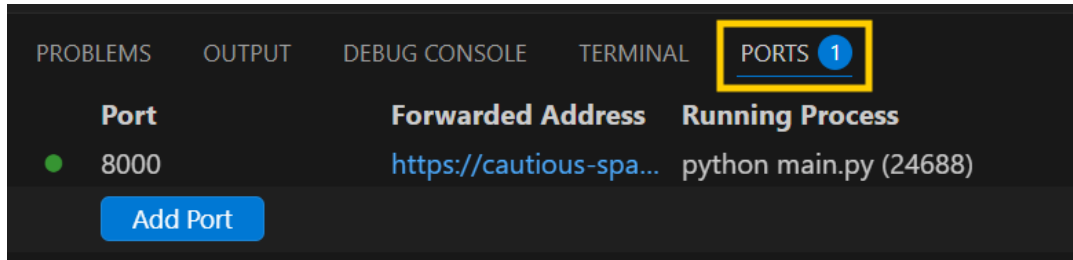
# Start application
uvicorn.run(app)
```

14. Start the application using below command:

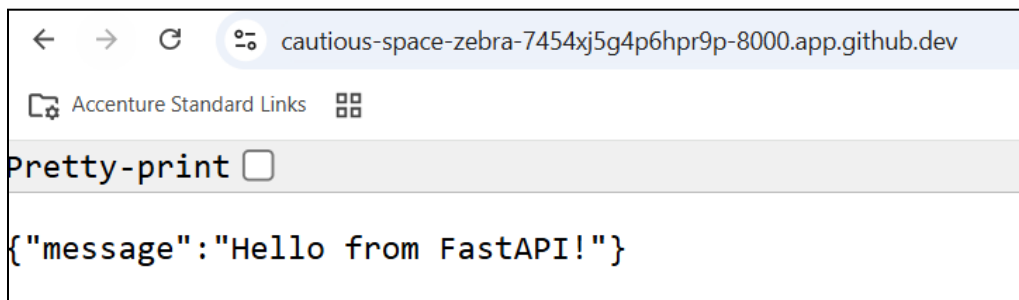
```
python main.py
```

```
(venv) @yrajm1997 →/workspaces/codespaces-blank $ python main.py
INFO:     Waiting for application startup.
INFO:     Application startup complete.
INFO:     Uvicorn running on http://127.0.0.1:8000 (Press CTRL+C to quit)
INFO:     223.181.42.187:0 - "GET / HTTP/1.1" 200 OK
INFO:     223.181.42.187:0 - "GET /favicon.ico HTTP/1.1" 404 Not Found
█
```

15. To access the application, go to the **Ports** tab (showing beside the Terminal tab) in Codespace >> Hover over the **Forwarded Address** and select **Open in the Browser** icon.



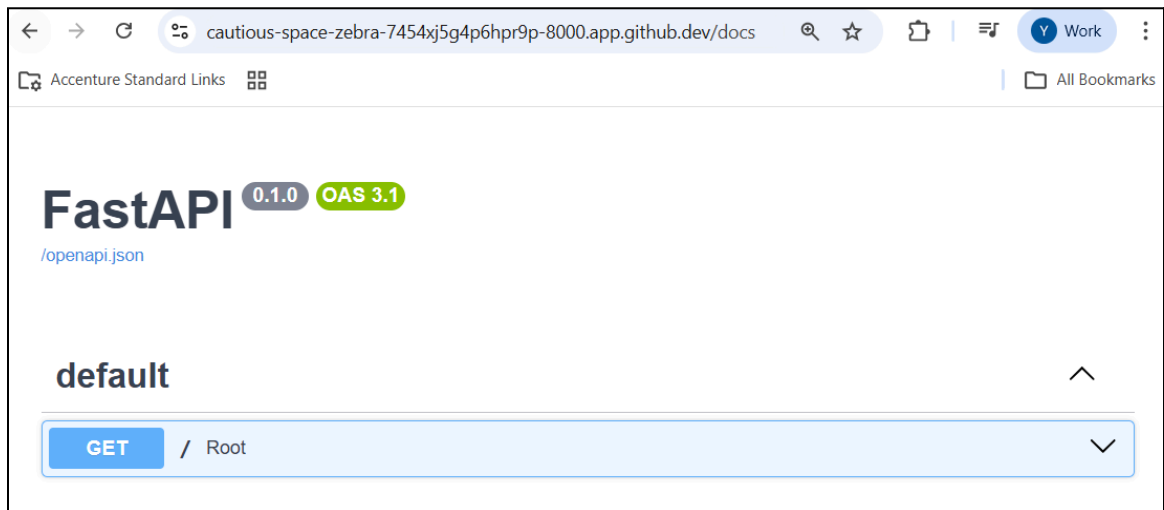
The application should be accessible.



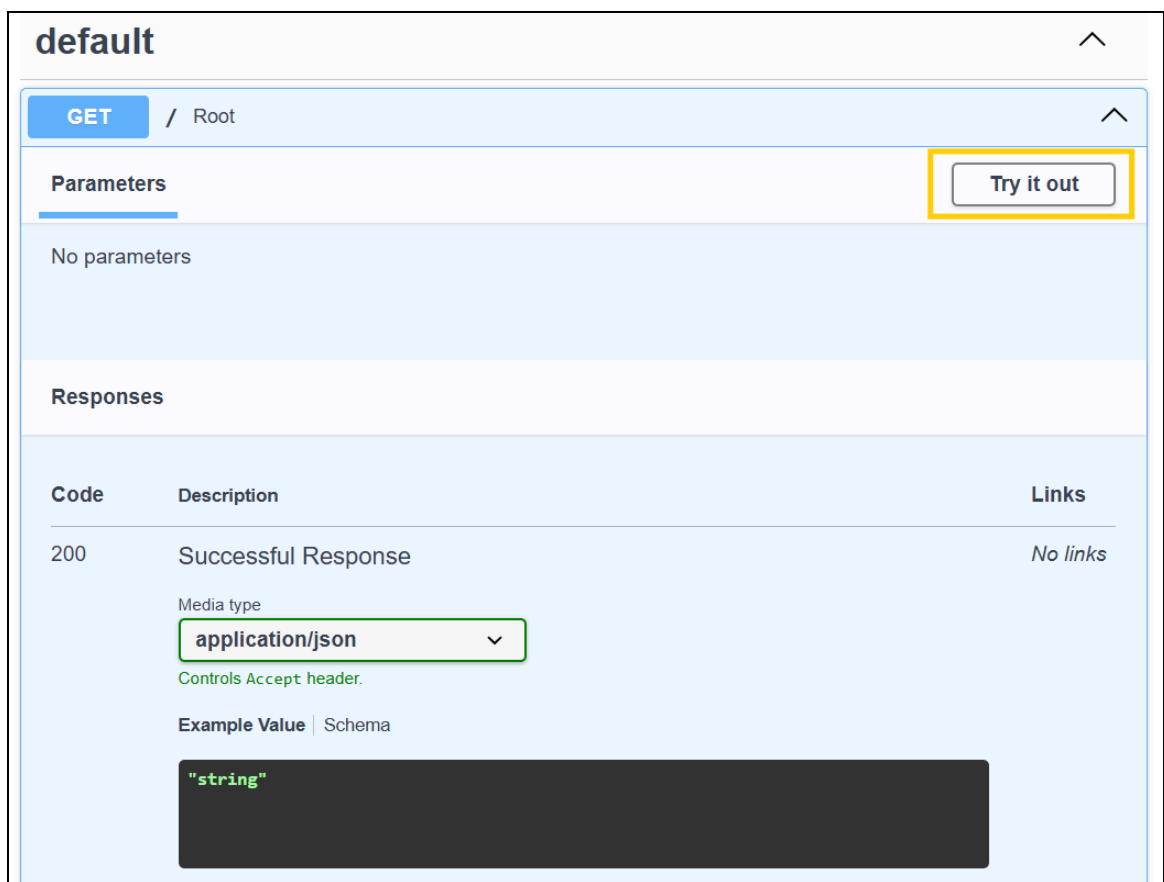
Automatic API Documentation

FastAPI provides automatic, interactive API documentation (with endpoints, schemas & authorization details) out of the box, leveraging Python type hints and the OpenAPI standard. This feature requires no extra configuration for basic usage and dynamically updates as you change your code.

16. Once the application is up and running, access the API documentation / Swagger UI at the “/docs” endpoint.



17. To test the endpoints from Swagger UI, just expand the particular endpoint and select **Try it out** >> **Execute**



Response should be visible.

Responses

Curl

```
curl -X 'GET' \
  'https://cautious-space-zebra-7454xj5g4p6hpr9p-8000.app.github.dev/' \
  -H 'accept: application/json'
```

Request URL

```
https://cautious-space-zebra-7454xj5g4p6hpr9p-8000.app.github.dev/
```

Server response

Code	Details
200	Response body <pre>{ "message": "Hello from FastAPI!" }</pre>  Download

Creating Basic Endpoints

Create basic GET endpoints with Query and Path parameters.

18. Replace the existing **main.py** file with below content.

Python

```
# Import FastAPI
from fastapi import FastAPI

# Create FastAPI object
app = FastAPI()

# Add a GET method to the "/" endpoint
@app.get("/")
def root():
    return {"message": "Hello from FastAPI!"}

# Define a GET API endpoint "/greet" with Query parameters
# In Browser, access the endpoint like this:
"/greet?first_name=Amar&last_name=Kumar"
```

```
@app.get("/greet")
def search(first_name: str, last_name: str):
    msg = "Greetings! " + first_name + " " + last_name
    return {"message": msg}

# Define a GET API endpoint "/hello" with Path parameters
# In Browser, access the endpoint like this: "/hello/Amar/Kumar"
@app.get("/hello/{fname}/{lname}")
def get_item(fname: str, lname: str):
    msg = "Hello! " + fname + " " + lname
    return {"message": msg}

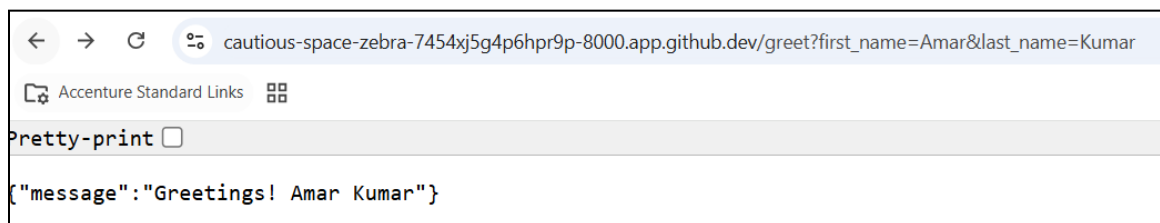
# Import Uvicorn (Web server)
import uvicorn

# Start application
uvicorn.run(app)
```

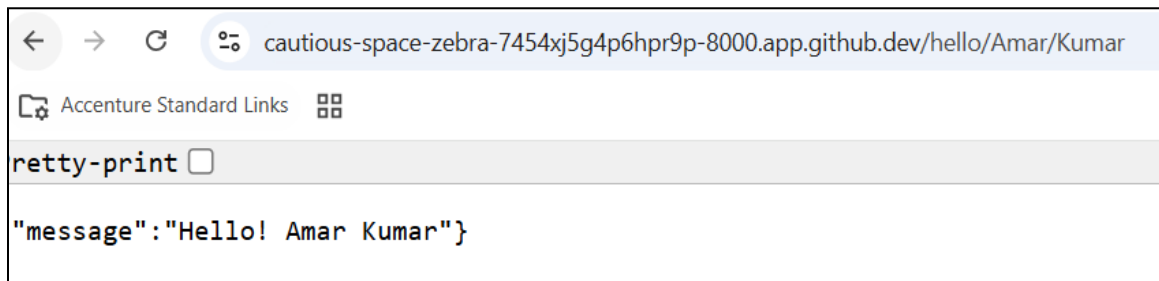
19. In the Codespace's Terminal, press Ctrl+C to stop the application. Then start it again
`python main.py`

Access the endpoints and provide the parameter values.

`"/greet?first_name=Amar&last_name=Kumar"`



`"/hello/Amar/Kumar"`



Handling Request Bodies (POST method) with Pydantic Models

When you need to send data from a client (let's say, a browser) to your API, you send it as a **request body**. A **request body** is data sent by the client to your API. A **response body** is the data your API sends to the client.

Your API almost always has to send a response body. But clients don't necessarily need to send request bodies all the time, sometimes they only request a path, maybe with some query parameters, but don't send a body.

To **declare a request body**, **Pydantic models** are used. FastAPI uses **Pydantic models** to validate and parse JSON bodies.

20. Replace the existing **main.py** file with below content.

Python

```
# Import FastAPI
from fastapi import FastAPI

# Create FastAPI object
app = FastAPI()

# Add a GET method to the "/" endpoint
@app.get("/")
def root():
    return {"message": "Hello from FastAPI!"}

# Define a GET API endpoint "/greet" with Query parameters
# In Browser, access the endpoint like this:
# /greet?first_name=Amar&last_name=Kumar
@app.get("/greet")
```

```
def search(first_name: str, last_name: str):
    msg = "Greetings! " + first_name + " " + last_name
    return {"message": msg}

# Define a GET API endpoint "/hello" with Path parameters
# In Browser, access the endpoint like this: "/hello/Amar/Kumar"
@app.get("/hello/{fname}/{lname}")
def get_item(fname: str, lname: str):
    msg = "Hello! " + fname + " " + lname
    return {"message": msg}

# Import BaseModel from Pydantic to define structured request body models
from pydantic import BaseModel

# Define a data model (schema) for validating incoming JSON data
class Person(BaseModel):
    fname: str    # First name (must be a string)
    lname: str    # Last name (must be a string)
    age: int      # Age (must be an integer)

# Create a POST API endpoint at "/person"
@app.post("/person")
def add_person(p: Person):
    # 'p' automatically receives and validates the JSON body using the
    # Person model

    # Construct a personalized message using the received data
    msg = "Hi! " + p.fname + " " + p.lname + ". Your age is " +
    str(p.age)

    # Return the message as a JSON response
    return {"message": msg}
```



```
# Import Uvicorn (Web server)
import uvicorn

# Start application
uvicorn.run(app)
```

21. In the Codespace's Terminal, press Ctrl+C to stop the application. Then start it again
`python main.py`

POST endpoints cannot be opened directly in a browser. To test them, use the `/docs` endpoint.



The image shows the FastAPI 0.1.0 OAS 3.1 API documentation interface. It features a header with the FastAPI logo and version information. Below the header, there is a section titled "default" with a list of endpoints. The endpoints are:

- GET / Root
- GET /greet Search
- GET /hello/{fname}/{lname} Get Item
- POST /person Add Person

The POST endpoint is highlighted in green, indicating it is the selected endpoint for testing.

Select Try it out >> Provide the input values in the request body >> Execute.

POST /person Add Person

Parameters Cancel Reset

No parameters

Request body required application/json

Edit Value | Schema

```
{  
  "fname": "Amar",  
  "lname": "Kumar",  
  "age": 30  
}
```

Response:

Server response

Code	Details
200	Response body <pre>{ "message": "Hi! Amar Kumar. Your age is 30" }</pre>Download